Migration status

Updated: 2026-10-06 · **Phase:** commit-only migration executed from scratch (343 single-subject commits, local `smp` branches, nothing pushed) and verified with the full xPack test run. Five repositories: the kernel (`micro-os-plus-iii`) and four architecture ports (`cortexm`, `aarch32`, `aarch64`, `posix-arch`). The devices library (`micro-os-plus-iii-devices`) is dissolved into the ports and retired.

This file is the cold-start entry point. Read it, then `docs/specs/2026-09-20-micro-os-plus-iii-smp-unification-design.md` for the full design and the measurements behind it. The sections after *Today* are the migration log, dated where they were measured.

Today (2026-10-06)

The migration from `xpack-development` (single CPU) to `smp` (SMP) was rebuilt from scratch with **commits only**, as described in `docs/smp-integration/xpack-dev-smp.md` Part II. Its Part III holds the step summary, the execution record and the complete commit list.

- **Source and method.** Fresh clones of `github.com/dan-maio/*` in `/tmp`, with `~/Work2/micro-os-plus/*` as a read-only `golden` remote. Every commit covers one subject, only adds or modifies files, and becomes one pull request (local branch `pr/<ID>`). No `scripts/smp/` and no chunk recipes are committed.
- **Commits.** kernel 125 (+ K985, Part III of xpack-dev-smp.md), posix-arch 24, cortexm 86, aarch32 64, aarch64 43, devices 1 (retirement README, kept in a bundle) — 343 in all.
- **Devices dissolved.** The SD, flatfs, DWC2 USB and FatFs drivers are in `drivers/`, and the SoC support in `soc/<chip>/`, of each port that uses them: posix-arch `soc/native`; cortexm `soc/stm32f4xx`, `soc/rp2350`; aarch32 `soc/bcm2837`, `soc/rk3506`; aarch64 `soc/bcm2837`. `UOS_DEVICES_DIR` no longer exists; the ports define `micro-os-plus::devices*` / `soc-*` themselves.
- **Built at every commit** (`xpm run build`), except the members of the two groups that only build together (G-hrclock, G-harness), the add-only aarch port commits before their first board, and the legacy hardware platforms `nucleo-*` / `raspberrypi-pico`, whose configure fails in golden as well.
- **Parity.** Identical to golden `smp` except the intended differences listed in xpack-dev-smp.md §21.4.
- **Full test run (latest compilers):** `install-all` rc 0; `test-native-cmake-sys` 16/16 (debug, release); `test-cortex-cmake` 3/3 in all 8 configurations; `test-smp-cmake` — aarch32 and aarch64 rpi-zero-2w/rpi3b 15/15, `2xcortex-m33` 4/4, `pico2-1cpu` 4/4, `cortexm-pico2` 6/6, `cortexm-pico2-rp2350b-psram` 3/3, each in debug and release.
- **Added since: `cortexm-pico` (Raspberry Pi Pico, RP2040, 2× Cortex-M0+), 2026-10-13.** No Pico SDK. cortexm gains `soc/rp2040`, the RP2040 port core (`src/rtos/os-core-rp2040.cpp`, `include-rp2040/`), the `pico` board and `test/pico/` (the pico2 tests, adapted); the kernel repo gains `tests/platforms/cortexm-pico` and its configs. The DHCSR “debugger attached” check is skipped on ARMv6-M (M4/M33 unchanged). Debug: 6/6 QEMU, 20/20 on the board through the JTAGprobe (SWD, 15 MHz). Release: 6/6 QEMU, 18/18 hwd (the USB tests not run).

The sections below are the earlier log. Where they mention `micro-os-plus-iii-devices`, `UOS_DEVICES_DIR` or `scripts/smp/`, read them as history: those no longer exist on the rebuilt `smp` branches.

Today (2026-10-01)

The concrete, executable **SMP upstream-integration runbook** is written and has been **dry-run from end to end — all 30 steps — entirely on local branches, with nothing pushed to any remote.** See `docs/smp-`

`integration/Implementation-SMP-Integration.md` (`.pdf`); its new §11 is the plain-English playbook and the repeatability contract.

- **Tooling** lives in `scripts/smp/` (bootstrap, new-step, verify-step, check-pristine, advance-step, link-ports, release-port, finalize, plus the per-step recipes in `chunks/` and the shared asserting-editor `chunks/_edit.py`). Every chunk reproduces its change byte-for-byte from `origin/smp`; every surgical edit asserts its anchor, so a moved source fails loudly instead of silently.
- **What ran green (FAST gate, xPack-pinned toolchains):** Part A 1–13; Part B collapsed (single-core invariant holds; native gcc14 debug+release 100%); Part C — port releases `posix-arch v1.1.0` / `cortexm v1.2.0` (local tags only), the **integrated** modular CMake bringing **dual-core Cortex-M33 SMP on QEMU mps2-an521** (3/3, then 4/4 with the new `fp-switch` FPU-switch test), and a `native-smp` platform running **genuine dual-core host SMP** (3/3); Part D — the final merge reconciles with baseline with no conflicts and the frozen metadata (`.github`, `README`, `LICENSE`) stays byte-identical.
- **Corrections captured during the dry run (all folded back into the recipes):**
  1. the `devices` repo is dissolved into the arch ports (Part 0) — the integrated cortexm links the fat `micro-os-plus::iii` and takes **no** `devices` sibling;
  2. `OS_USE_SMP_SCHEDULER` must be set via a platform's `target_compile_definitions`, not a `cmake -D` **cache variable** — the latter secretly builds single-core and was masking a real kernel `port_cpu_id` redundant-decls defect (now fixed); (3) `npm version` runs a `postversion` push hook, so `release-port.sh` edits the version in-place instead; (4) the per-step `perl` edits were replaced with an asserting Python editor.
- **Mandatory full-matrix gate — 16/16.** `scripts/smp/full-verify.sh` drives the real VS-Code-equivalent xpm flow (`install → link-deps → prepare → build → test`) for every platform with one gcc + one clang: `native` (gcc14+clang19), `native-smp`, `qemu-cortex-m0/m3/m4f/m7f`, `2xcortex-m33`, debug+release — all green. This replaced the earlier FAST-only "green", which had left ~58 of ~60 configs unbuilt and hid two real failures: a released-vs-integrated **port** mismatch (`qemu-cortex-m7f`) and **clang `-Weverything`** breakages — both now fixed in the recipes (`part-b.sh` fixes I/J, `step03.sh`, `step05.sh`).
- **Newer host toolchains.** The tests also build + run on compilers newer than the pinned ones — verified system **gcc 16.2.1** and **clang 22.1.8** (native, single- and dual-core SMP, 100%). clang 22 needed three baseline/upstream compat fixes (`estd/chrono` libc++-internal rename, two new clang-20+ diagnostics, and a portable unwinder choice) — captured in `scripts/smp/newer-toolchains.sh`, separate from the SMP lift (see runbook §3.2).
- **cortexm-pico2 (RP2350) added — harness verified on QEMU.** New add-only platform `tests/platforms/cortexm-pico2` + build configs. Following the smp branch's own design, the *emulated* pico2 image uses the generic Cortex-M port
  - `device-qemu-cortexm` (QEMU `mps2-an500`, cortex-m7), on which the portable harness suites run: **rtos-apis + mutex-stress + cmsis-os-validator (60/60) = 3/3, 100%** (`step-31-green`). The RP2350-specific `smp-test0..5` are **not** added yet: they need the real RP2350 board BSP (SIO hardware spinlocks at `0xD0000100`, 2-core boot, PL011/multicore glue), which is hardware-only — there is no QEMU machine for RP2350, so those tests can be build-verified at best and never emulated here. aarch32/aarch64 (rpi3b / rpi-zero-2w) are the same shape (standalone board BSPs) and remain to be re-hosted.
- **aarch64-rpi3b SMP integrated from scratch — all 5 smp tests PASS on QEMU (4-core).** The aarch64 port existed only on `smp` as a 100-file standalone-model port (its `xpack-development` branch is an empty stub). It was integrated into the xpack topology on local `step/32`: the port (context_switch, exception_handler, handlers, smp_secondary, os-core), the A53 BSP (startup.S, MMU, GIC, PL011, ARM generic timer, **spin-table secondary release**, linker-rpi3b.ld), the **BCM2837 SoC dissolved out of `devices`** (Part 0), the 37-file **thin** kernel (fat `::iii` doesn't apply — the port owns trace/startup; kernel `trace::write` is `weak`), plus a port-supplied `free-store.cpp` (the kernel's is `__ARM_EABI__`-only) and a `_sbrk` syscall stub. Result on `qemu-system-aarch64 -M raspi3b -smp 4` (aarch64-none-elf 15.2.1, semihosting, `-DQEMU_BUILD`): **smp_test0 (core-0 bring-up), smp_test1 (4-core semaphore ping-pong, `join: c1=3 c2=3 c3=3`), smp_test2, smp_test3, smp_test4 — all RESULT: PASS. aarch64-rpi-zero-2w passes identically** (same BCM2837/A53 BSP, `linker.ld`, `LED_PIN=29`): smp_test0-4 all PASS on QEMU raspi3b 4-core. Both aarch64 boards are captured in the reproducible `scripts/smp/build-aarch64-rpi.sh`

(`BOARD=rpi3b|rpi-zero-2w`), which compiles the thin kernel + port + BSP + SoC + free-store/syscalls, links, and runs each test on QEMU — 5/5 PASS each. The kernel also gained three additive CMake targets (`micro-os-plus::iii-core` thin, `::port-smp-decls`, `::test-support`) for the silicon ports.

- **aarch32 (ARMv7-A) done too — all four aarch boards green.** The aarch32 port (ARMv7-A, arm-none-eabi, `-marm -mcpu=cortex-a53 -mfpu=neon-fp-armv8`) was integrated the same way; being `__ARM_EABI__` it reuses the kernel's own free-store + `_sbrk` (no port copies needed), and it reaches the A53s through a tiny AArch64→AArch32 `shim8.img` (built from `qemu-raspi3-shim/shim.S`) + `-device loader,addr=0x10000`, because raspi3b starts its cores in AArch64. Reproducible via `scripts/smp/build-aarch32-rpi.sh` (`BOARD=rpi3b|rpi-zero-2w`). **Full matrix — each 5/5 smp_test0-4 PASS on QEMU raspi3b, 4-core SMP:** aarch64-rpi3b, aarch64-rpi-zero-2w, aarch32-rpi3b, aarch32-rpi-zero-2w. Remaining polish: fold these builds into harness CMake platforms+configs and add the per-board harness suites (rtos-apis/mutex-stress/cmsis-os-validator).
- **Not pushed. Not published.** The whole series is local `step/NN` branches + `step-NN-green` tags; `git ls-remote` shows zero of them on any origin. Publishing is a deliberate manual `git push` that no script performs.

Today (2026-09-29)

The SMP repository rebase and xPack test framework restoration have been completed and verified across all target architectures:

- **Repository unification.** `micro-os-plus-iii-smp` was rebased on top of `micro-os-plus/micro-os-plus-iii` on branch `smp`. Sibling repositories now live under `/home/dan/Work/micro-os-plus/`: `micro-os-plus-iii`, `micro-os-plus-iii-cortexm`, `micro-os-plus-iii-aarch32`, `micro-os-plus-iii-aarch64`, `micro-os-plus-iii-devices`, and `micro-os-plus-iii-posix-arch`. CMake sibling discovery resolves directories both with and without `.git` suffix (`micro-os-plus-iii-<port>` and `micro-os-plus-iii-<port>.git`).
- **xPack test matrix restoration.** Restored the complete xPack test system structure in root `package.json` and `tests/package.json` with 35+ configurations. Automated testing (`xpm run test --config C`) runs `ctest -V -LE hwd`, executing emulated and host tests while excluding hardware tests.
- **Pico 2 RAM targets.** Added pure-RAM resident test targets (`smp-mat-test-ram`, `rtos-apis-ram`, `mutex-stress-ram`, `cmsis-os-validator-ram`) executing directly from internal SRAM via OpenOCD without flash writes (26 tests total on `cortexm-pico2`).
- **Code bugfix in `hw_result.hpp`.** In `micro-os-plus-iii-cortexm/test/boards/shared/hw_result.hpp`, changed `std::exit(0)` to `std::_Exit(0)`. Normal `std::exit()` invokes `atexit` handlers and static object destructors while RTOS threads are still running, causing `assert(empty())` assertion failures in `double_list::~double_list()`. `std::_Exit(0)` terminates execution cleanly without running static destructors.
- **Runner extension fallback.** Added `.elf` file extension handling in board runner scripts (`hw.sh`) to support varied toolchain output conventions.
- **AArch64 Raspberry Pi verification.** 30/30 tests PASS under QEMU `raspi3b -smp 4` across `aarch64-rpi-zero-2w` (15/15) and `aarch64-rpi3b` (15/15).
- **Pico 2 / Cortex-M33 verification.** 16/16 emulated tests PASS:
  - `pico2-1cpu-cmake-gcc-debug`: 4/4 PASS on QEMU `mps2-an505`.
  - `2xcortex-m33-cmake-gcc-debug`: 4/4 PASS on QEMU `mps2-an521`.
  - `cortexm-pico2-cmake-gcc-debug`: 6/6 PASS on QEMU `mps2-an500`.
  - `cortexm-pico2-rp2350b-psram-cmake-gcc-debug`: 2/2 PASS on QEMU `mps2-an500`.
- **STM32F4 (Cortex-M4F) verification on QEMU `netduinoplus2`.**
  - `cortexm-nucleof411`: 4/4 PASS on QEMU `netduinoplus2 -cpu cortex-m4` (`mos-test1`, `rtos-apis`, `mutex-stress`, `cmsis-os-validator`).
  - `cortexm-weactf411`: 5/5 PASS on QEMU `netduinoplus2 -cpu cortex-m4` (`mos-test1`, `spi-pipeline`, `rtos-apis`, `mutex-stress`, `cmsis-os-validator`).

    - `cortexm-weactf412`: Confirmed hardware-only due to 256 KB SRAM requirement (`_estack` at `0x20040000`, exceeding QEMU Netduino's 128 KB memory map) and unsupported `DMA2_Stream7` in `uart-test1`.
- **posix-arch: the CPU id is never read through a cached thread pointer.** clang 16–18 at `-O2` computed `&_this_cpu` once per function and kept it in a register across `reschedule()`; a thread that resumed on another host thread then took the kernel lock as the CPU it had left, two CPUs ran inside it, and the lock was stranded held (`owner 0, depth 1`, every other CPU spinning). `smp_test3` and `smp-pipeline-test` hung on every run under `native-cmake-clang{16,17}-release`; clang 18 had the same code in 62 kernel functions and passed by timing. `port_cpu_id_inline()` now calls the out-of-line `port_cpu_id()`, which is `noinline` with an `asm volatile` barrier. GCC and clang 19 were never affected. See posix-arch-port.md §2, "Native TLS and migration".

2026-09-26

A review of the SMP code found ten defects; all ten are fixed, each with a test that failed first where one could be written.

- **Kernel lock bookkeeping (all SMP ports).** `scheduler::locked()` read the core id before masking interrupts, so a migration between the read and the mask left the lock owned from one core and marked on the other; the unlock then released nothing. Fixed on RP2350 and M33 (cortexm `f000383`) and POSIX (posix-arch `040bb09`). The RP2350 unlock also publishes "unlocked" before releasing the spinlock, as the M33 port does.
- **`this_thread::thread()` (kernel `60b55a3`)** read `current_thread_[core]` with interrupts enabled; a migration between the two reads returned another core's thread.
- **FPU context on the Cortex-M SMP ports (cortexm `92caefe`).** ASPEN/LSPEN were off and PendSV saved no FP state; now s0–s15/FPSCR are stacked lazily and PendSV saves s16–s31. New suite `fp-switch` (kernel `c41c06d`), on `cortexm-pico2`, `2xcortex-m33` and `pico2-1cpu`.
- **Mutex protect ceiling (kernel `a734798`).** A lock refused with `EINVAL` left the mutex on the thread's owned list; destroying the thread then marked it owner-dead. Inherited from upstream. Test: native `mutex-ceiling-test`.
- **flatfs (devices `ee58885`).** `append_file()` on a file created empty wrapped its extent and wrote from sector 0. Test: native `flatfs-test`.
- **RP2350 SIO spinlocks (cortexm `2e679ad`)** are released at reset; a run that ended holding the kernel lock hung the next image (`smp-mat-test` on the psram board).
- **SMP `thread::kill()` synchronization.** On SMP, `thread::kill()` can wait forever on a target thread executing on another core that keeps picking it. Now `thread::kill()` triggers an immediate cross-core reschedule IPI (`port_smp_ipi(busy_cpu)`). To eliminate double-destruction races with the idle thread reaper (`os_rtos_idle_actions()`), `state::destroying = 7` was added to arbitrate ownership under `interrupts::critical_section`.
- **Monotonic SMP `hrclock` (`clock_highres::now()`).** Replaced SysTick subdivision (which mixed core 0's tick count with the calling core's SysTick phase) with cluster-wide monotonic hardware counters queried lock-free via `if constexpr (port::clock_highres::has_hardware_counter())`. Implemented on AArch64 (`CNTPCT_EL0`), AArch32 (`CNTPCT`), RP2350 (`TIMER0` 1 MHz latch), and POSIX (`CLOCK_MONOTONIC`), with fallback to SysTick on generic Cortex-M.
- **`hrclock` consistent with its own sleeps (2026-10-22).** The hardware counters above were raw: `now()` counted from power-on (AArch64, AArch32), from TIMER0's reset (RP2350) or from the host's boot (POSIX), while `hrclock`'s sleeps compare with the tick-driven steady count, from the kernel's start – `hrclock.sleep_until (now () + 100 ms)` slept 391 ms (rpi4b, QEMU), and on POSIX the host's uptime. Each port now subtracts its tick origin. Also: the AArch tick is re-armed on an absolute grid (`CNTP_CVAL += period`; re-arming one period from now lost the handler's latency every tick), the counter's frequency is calibrated once instead of by every core, the RP2350's `cycles_per_tick()` is in TIMER0 microseconds (it was CPU cycles, ~150 times too much) and its ticks are caught up from TIMER0. New board test `clock-test` (rpi4b, pico2, and the Zero 2 W and Pi 3 B of both ports), PASS on the Pi 4, the Pico 2 and the Zero 2 W (aarch64 and aarch32); the Zero 2 W and Pi 3 B also take a CMSIS-DAP probe (`PROBE=cmsis-dap`).

- **The skipped ticks move `hrclock`'s origin (2026-10-22, aarch32).** The absolute re-arm skips the ticks of a halt longer than 1000 periods (a debugger, or a slow semihosting call: the counter keeps running) instead of replaying them; core 0's skipped ticks now move `hr_base` too. On the Lyra the start-up's semihosting halts over the WCH-Link skipped 8.9 s of ticks, and `hrclock.now()` stayed that far ahead of its steady count: `hrclock.sleep_until (now () + 100 ms)` took 8.98 s. `clock-test` added to the Lyra; it passes there with the change. (The same resync is in the aarch64 ports, not changed yet.)
- **Duplicate gate repo and path normalization.** `verify-no-duplicate-sources.py` now normalizes repository directory names (stripping trailing `.git` and resolving symlinks) and sibling comparison paths so canonical repo names and sibling exemptions match accurately across the multi-repo workspace.
- **Tests.** pico2's HID test pins its USB thread at construction (`1e97fbf`). The psram nested tests end after 20 beats with a verdict (`71a717b`). The Lyra USB tests enumerate and end with a verdict, their console UART-only, their host tools as source in each test's `host/` (aarch32 `b9dada8`); the Lyra's `smp-pro-cons-test` pins to cores it has (`d173d0e`).
- **Applications.** `pico2` 16 (`fp-switch`), `native` 15 (`flatfs-test`, `mutex-ceiling-test`); the rest as below.
- **Verified.** On the boards: `pico2` 16/16, `pico2-rp2350b-psram` 14/14, the Lyra's four USB tests and `smp-pro-cons-test`. Under QEMU and native: every one of the 38 local configurations builds and its set passes, run one configuration at a time. All code strictly C++20.

2026-09-25

- **Workspace.** The working copies are `~/Work/micro-os-plus-iii-*.git`, clones of the bare repositories in `~/Downloads/GIT/`. The migration source, `TMP7/micro-os-plus-iii-smp-old`, is still the read-only reference.
- **One test framework.** Everything is registered and run through the xPack harness in `micro-os-plus-iii-smp.git/tests/`: 22 platforms, 70 configurations (35 debug + 35 release). How to use it is `tests/STEPS.md`; every test, what it does, its scheduling mode and each board's probe is `tests/TESTS-CATALOG.md`.
- **Boards and applications.** aarch32: `rpi-zero-2w` 15, `rpi3b` 15, `luckfox-lyra` 19. aarch64: `rpi-zero-2w` 15, `rpi3b` 15. cortexm: `nucleof411` 4, `weactf411` 5, `weactf412` 5, `pico2` 15, `pico2-rp2350b-psram` 14, `pico2-pizero` 14. posix-arch: `native` 13, plus the harness's `cmsis-os-validator`.
- **The harness suites on the boards.** `mutex-stress`, `rtos-apis` and `cmsis-os-validator` are board applications on the four Raspberry Pi platforms, on `pico2` and on the three STM32 boards; the Lyra runs `mutex-stress`.
- **The CMSIS-RTOS validator on the Cortex-A53.** It passes 60/60 on the four Raspberry Pi platforms (Zero 2 W and Pi 3 B, AArch32 and AArch64), under QEMU and on the boards, debug and release. Its NVIC IRQ 0 is the BCM2837's local Mailbox 1 and its DWT probe hits a "no cycle counter" page, both only in the validator's image (`UOS_CMSIS_OS_VALIDATOR`); every other image is byte-identical to before.
- **native.** All 14 host cases pass in `native-cmake-{gcc,sys}-{debug,release}`.
- **The gates, re-run today.** `verify-no-absolute-paths.sh` passes. `verify-no-duplicate-sources.py` had been reporting 54 unexplained pairs, most of them because it matched repositories by name and the workspace's directories carry `.git`: its exemptions (the vendored CMSIS headers) and every `SIBLINGS` entry silently stopped applying. With that fixed, the real duplicates were removed (one verdict wrapper per Cortex-M board instead of one per suite; pico2's QEMU `clocks.hpp` copy) and the genuine look-alikes named in `SIBLINGS` with a reason each; every image rebuilt byte-identical. The last pair was cortexm's `include/` and `include-rp2350/` `os-decls.h`, code-identical since the STM32 core gained its OS_NCPU=1 SMP branch (`04940fe`). The RP2350 copy is gone: pico2's `UOS_BOARD_PORT_INCLUDE` is now `include-rp2350;include`, so `include-rp2350/` holds only the two headers that differ. The gate **passes**. The STM32, `2xcortex-m33` and `pico2-1cpu` images rebuilt byte-identical; the pico2-family images are identical apart from the build date and time their banners print (`__DATE__`/`__TIME__`), which change on every relink. `verify-kernel-compiles.sh` now takes a `:`-separated list of include directories: `posix-arch` 61/61; `cortexm` 27/61 on `include/`, `include-m33/` and `include-rp2350:include` alike (the RP2350's old standalone directory also gave 27).

- **The kernel-compiles gate reaches the AArch ports.** It compiles `.c` as C, takes the port's cross compiler plus extra flags (the board facts), and `--without <group>` skips an optional group the port does not link. With those, `aarch64` and `aarch32` both **PASS**; see the gate table below for the exact commands.

Where everything stood on 2026-09-22, measured

Re-run from a clean tree at the head of every repository, one QEMU suite at a time on an idle host:

what	result
`aarch32` / `rpi-zero-2w`, QEMU	11 passed, 1 skipped, 0 failed
`aarch32` / `rpi3b`, QEMU	11 passed, 1 skipped, 0 failed
`aarch64` / `rpi-zero-2w`, QEMU	11 passed, 1 skipped, 0 failed
`aarch64` / `rpi3b`, QEMU	11 passed, 1 skipped, 0 failed
`posix-arch` / `native`, host	11 passed, 0 skipped, 0 failed
`posix-arch` / `native`, under ASan	11 passed, 0 skipped, 0 failed
`posix-arch` / `native`, under UBSan (`NO_VPTR`)	11 passed, 0 skipped, 0 failed
every build directory — both Pi boards × 2 ports, the Lyra, 4× `cortexm`, the host	builds

`usb_test` is the skip, by design: QEMU emulates no USB device mode.

Nothing in this workspace is failing a test, and all three gates pass.

Steps 1 and 2 are complete. Six repositories exist; both A-profile architecture projects build all 24 of their targets from a single copy of every test, and all twelve tests pass on a Raspberry Pi Zero 2 W on **both** ports — each measured on that port, neither inferred from the other.

Step 3 is in progress. The AArch32 port now carries a second board, the Luckfox Lyra B (RK3506, 3× Cortex-A7): 19 targets build, and both ports still pass their QEMU suites (11/1/0 each) after the restructure it required. What that second board changed, and the seven portability defects it exposed in shared code, are written up in `aarch32-second-board.md`.

The Lyra is a hardware-only board, decided and closed. QEMU models none of the RK3506's own blocks, so an emulated Lyra would be a generic Cortex-A7 wearing the name — and the DWC2 gadget, the SD host and the Cortex-M0 mailbox, which are what make this board worth having, are exactly the parts not modelled. Its `board.cmake` sets no `UOS_BOARD_LINKER_QEMU`, so it builds one image per test rather than two, and `test/qemu.sh` says so if asked. This is not a gap to close later; do not reopen it.

Step 5 is complete. `micro-os-plus-iii-posix-arch` exists: SMP on the development machine, one host thread per CPU, preemptive. One board, `native`, eleven tests, **11 passed / 0 skipped / 0 failed** — nine SMP object tests carried unchanged at `OS_NCPU=4`, and `mutex-stress` and `rtos-apis` from the kernel's own `tests/sources/` at `OS_NCPU=1`. It is the only one of the four ports that is new work rather than a migration, and it has now earned its keep twice: it found a real SMP race in the kernel's mutex (see below), and, the first time it was run under ASan, a dangling thread name that has shipped in every copy of `smp-pro-cons-test` since the test was written (see *Sanitizers* below). Written up in `posix-arch-port.md`.

Step 4 is under way. `micro-os-plus-iii-cortexm` exists, with six boards: three STM32F4 at `OS_NCPU=1` and three RP2350 boards at `OS_NCPU=2` — the Raspberry Pi Pico 2, the WeAct RP2350B with 8 MB of PSRAM, and the Pi-Zero RP2350B. The STM32 boards are the first `OS_NCPU=1` boards in the workspace, so they are also the first to build the kernel's **non**-SMP branch — every board before them had more than one CPU. Nothing on this port has run on hardware yet. The port, its six boards and where its SMP lives are written up in `cortexm-port.md`.

```
~/Work/                           the workspace (application counts as of 2026-09-25)
├── micro-os-plus-iii-smp/        kernel + shared build rules + test runners
│   ├── src/ include/             the kernel, upstream path-for-path
│   ├── port/smp-common/          os-decls.h, shared by both ARM ports
│   ├── test_smpl/                the three runners -- see test-smpl.md
│   │   ├── run-qemu.sh           the shared QEMU suite runner
│   │   ├── run-host.sh           the same, for the POSIX host
│   │   └── run-hw.sh             the shared hardware session runner
│   ├── cmake/                    toolchains + uos_add_app
│   └── tools/                    the verification gates
│       ├── verify-kernel-compiles.sh        kernel builds on a port's headers
│       ├── verify-no-duplicate-sources.py   spec Section 9, cross-repo
│       └── verify-no-absolute-paths.sh      spec Section 9
│   (micro-os-plus-iii-devices — retired 2026-10-06: its drivers/ and
│    soc/<chip>/ now live in each port that uses them)
├── micro-os-plus-iii-aarch64/    ARMv8-A port
│   ├── src/ include/             ARMv8-A, every board
│   └── test/                     everything board- or test-specific
│       ├── CMakeLists.txt        one loop over test/<board>/*/
│       ├── hw.sh  qemu.sh        dispatchers; BOARD picks the directory
│       ├── boards/{rpi-zero-2w,rpi3b}/   board.cmake, include/, src/,
│       │                                 linker, OpenOCD, hw.sh, qemu.sh
│       ├── rpi-zero-2w/ rpi3b/   each board's test applications
│       └── build*/               the build trees
├── micro-os-plus-iii-aarch32/    ARMv7-A port
│   ├── src/ include/             ARMv7-A, every board — incl. the scheduler
│   └── test/                     same shape
│       ├── boards/{rpi-zero-2w,rpi3b,luckfox-lyra}/  BCM2837 · BCM2837 · RK3506
│       └── rpi-zero-2w/ rpi3b/ luckfox-lyra/         15 · 15 · 19 applications
├── micro-os-plus-iii-posix-arch/ POSIX host port — host threads as CPUs
│   ├── src/ include/             the CPU model, the scheduler half, the
│   │                             fault reporter, the free store
│   └── test/
│       ├── run.sh                dispatcher; BOARD picks the directory
│       ├── boards/native/        board.cmake, include/, src/, run.sh
│       └── native/               13 applications (11 at NCPU=4, 2 at NCPU=1)
├── micro-os-plus-iii-cortexm/    Cortex-M port — M4F and M33, 1 and 2 CPUs
│   ├── src/ include/             upstream's single-core core (STM32 boards)
│   ├── src/rtos/os-core-rp2350.cpp
│   │   include-rp2350/           its os-c-decls.h, os-inlines.h (pico2)
│   └── test/                     same shape
│       ├── boards/{nucleof411,weactf411,weactf412}/
│       │                         STM32F411RE · F411CE · F412RE
│       ├── boards/{pico2,pico2-rp2350b-psram,pico2-pizero}/
│       │                         RP2350 4 MB · RP2350B +PSRAM · RP2350B
│       │                         the last two include pico2's board.cmake
│       └── <board>/              4 · 5 · 5 · 15 · 14 · 14 applications
└── micro-os-plus-iii-smp-old/    READ ONLY — the migration source (in TMP7/, not here)

Outside `src/` and `include/` — which are the ISA and nothing else — an
architecture project is all `test/`: the boards, their tests and their build
trees. How that is laid out and how to run it is
[`tests-in-aarch32-aarch64.md`](tests-in-aarch32-aarch64.md).
```

Migration source (read-only)	`TMP7/micro-os-plus-iii-smp-old`
Remotes	`GIT/micro-os-plus-iii-{smp,devices,aarch32,aarch64,cortexm,posix-arch}.git`
Old remote	`GIT/micro-os-plus-iii-smp-old.git`

An architecture project holds **no copy** of the kernel. CMake resolves it as a sibling directory (`micro-os-plus-iii`, else `micro-os-plus-iii-smp`, with or without `.git`), overridable with `-DUOS_SMP_DIR=`. Since 2026-10-06 the drivers and SoC support are inside each port (`drivers/`, `soc/<chip>/`), so `-DUOS_DEVICES_DIR=` is gone. Whoever builds one clones what it needs. One working copy of the kernel serves every architecture project on the machine, so an edit to it is visible to all of them at once. The dependency runs one way: the kernel does not know an architecture project exists.

Done

1. **Remotes reorganised**, old tree analysed and measured, design spec written. Every figure in the spec came from the tree, not estimation.

2. **Step 1 — skeleton and kernel.** Kernel vendored once, flattened to the repository root so it matches upstream path-for-path. Three toolchains share one preamble. `uos_add_app` replaces 214 Makefiles.

3. **Devices split out** (`micro-os-plus-iii-devices`). All seven sources compile for armv7-a *and* armv8-a from one copy; ~12,000 duplicated lines collapse to one copy.

4. **Step 2a — the twelve test applications unified.** 15,507 lines of application C++ become 7,610. Nothing ISA-specific is left in a test.

5. **Step 2b — both ARM architecture projects.** Each builds every board's tests from one loop over `test/${BOARD}/*/`, in two variants. No test name and no board name appears in either port's build files. 24 targets per Pi board. The Lyra is 19: it is hardware-only, so it builds one image per test rather than two.

6. **QEMU suites green on all four board/port pairs.** Both ports × {`rpi-zero-2w`, `rpi3b`} give `11 passed, 1 skipped, 0 failed`, with gcc 15 and **one suite at a time**. `usb_test` skips by design — QEMU emulates no USB device mode — as the predecessor suite also recorded.

   `aarch32` + `rpi3b` was briefly recorded here as `10 / 1 / 1`, with `smp-mat-sdcard-test` TIMEOUTing after 2000 s. That was wrong, and the cause was procedural: the four suites had been launched two at a time, which is exactly what the "QEMU suites must run one at a time" rule below forbids, and the stall had that rule's documented signature — a worker frozen mid-loop with no fault. Run alone, that test passes in 113 lines, and the whole suite passes 11/1/0. **Re-run before investigating** is in that rule for a reason; it was not followed.

7. **Documentation.** `docs/smp-construction.md` (the port contract, the division of labour, and the defines per folder) and `docs/building-aarch32-aarch64.md`, both with PDFs, rendered by a single `docs/md2pdf.py` that replaces three near-identical copies.

8. **Hardware green on both ports.** All twelve pass on a Zero 2 W over a J-Link, `usb_test` included — the one QEMU can never run. `test_smpl/run-hw.sh` plus a ~40-line `test/hw.sh` per port replace the predecessor's 48 per-test runner scripts.

9. **Step 5 — the POSIX port.** A new SMP implementation, `OS_NCPU` host threads as CPUs, preemptive from the start: per-CPU `timer_create` + `SIGEV_THREAD_ID`, `pthread_sigmask` as per-CPU interrupt masking, `pthread_kill` as the IPI, the context switch performed inside the signal handler. Migration is allowed and native TLS is banned. 11/0/0.

10. **The two Section 9 gate scripts.** `verify-no-duplicate-sources.py` pools the tracked sources of *every* repository and compares them — running it one repository at a time would miss precisely the duplication D9's multi-repo layout permits. `verify-no-absolute-paths.sh` looks for machine-specific roots. Both were checked against injected violations, not only against a clean tree.

The three verification gates

None of these is a test. They are lints over the source tree, and they are recorded separately from the suites for that reason. A FAIL here means “the same file exists twice”, never “a test failed” — for most of this project’s life the duplicate gate reported FAIL while every suite on every board was green. Do not read the two tables as one.

gate	command	state
kernel compiles on a port’s headers	`tools/verify-kernel-compiles.sh ../micro-os-plus-iii-posix-arch/include`	PASS on `posix-arch`, `aarch64`, `aarch32`; `cortexm` 27/61 — see below
no duplicate sources, across repos	`tools/verify-no-duplicate-sources.py`	**PASS** — see below
no machine-specific absolute paths	`tools/verify-no-absolute-paths.sh`	PASS (105 files)

The kernel-compiles gate passes on three ports of the four (measured 2026-09-26; `$X` is `~/.local/xPacks/@xpack-dev-tools`). A bare-metal port needs its own cross compiler, because the host’s cannot assemble its inline asm, and the facts a board would supply follow it as flags:

port	command	result
`posix-arch`	`tools/verify-kernel-compiles.sh ../micro-os-plus-iii-posix-arch.git/include`	**PASS** 61/61
`aarch64`	`tools/verify-kernel-compiles.sh ../micro-os-plus-iii-aarch64.git/include:port/smp-common $X/aarch64-none-elf-gcc/15.2.1-1.1.1/.content/bin/aarch64-none-elf-g++`	**PASS** 61/61
`aarch32`	`tools/verify-kernel-compiles.sh --without startup ../micro-os-plus-iii-aarch32.git/include:port/smp-common $X/arm-none-eabi-gcc/15.2.1-1.1.1/.content/bin/arm-none-eabi-g++ -mcpu=cortex-a7 -marm -DOS_NCPU=3 -DOS_SMP_IPI_SGI=0 '-DPORT_GREETING="lyra"'`	**PASS** 57/61, 4 skipped
`cortexm`	`tools/verify-kernel-compiles.sh ../micro-os-plus-iii-cortexm.git/include` (also `include-m33`, `include-rp2350:include`)	27/61

- `aarch64` and `aarch32` take their shared `os-decls.h` from the kernel’s `port/smp-common/`, hence the two-directory path.
- `aarch32` refuses to guess `OS_NCPU`, `OS_SMP_IPI_SGI` and `PORT_GREETING` (`aarch64` has defaults for them), and its `cpsid`/`cpsie` need a Cortex-A `-mcpu`. With the rpi3b’s facts (`-mcpu=cortex-a53 -DOS_NCPU=4`) and no `--without`, it compiles 59/61. The two left are `startup/exception-handlers.c` and `startup/initialize-hardware.c`, which call the Cortex-M CMSIS `__disable_irq()` and `SystemInit()`. They belong to the optional `startup` group, which the AArch ports do not link (they have their own `startup.S`), so `--without startup` is the honest run.

- `cortexm`'s remaining sources need a board's vendor header (`cmsis_device.h`), so on this port the gate stops at the board.

The duplicate gate has **three** outcomes, not two, because this workspace duplicates some code on purpose and a flat pass/fail would have to lie about it:

- **exempt** — never compared: upstream's `tests/`, vendored `xpacks/`, TinyUSB, and ARM's own CMSIS core headers (429 files). One consequence is worth knowing: the two single-core tests the POSIX board carries, `mutex-stress` and `rtos-apis`, are near-copies of `tests/sources/`, and the gate cannot see the pairing because the originals are on the exempt side. It is deliberate — `tests/` is carried as shipped (spec Section 10) — but it is not the gate proving those copies are justified.
- **expected** — reported and counted, but not a failure: *the same test file carried by two boards*. Every board owns its tests, deliberately, so that changing a test reaches exactly one board. **256 pairs.** The spec's Section 9 says "target: zero findings" and was written before that decision; this is where the two are reconciled, in the open rather than by a silent exemption.
- **sibling** — reported, counted, and **named individually in the script's `SIBLINGS` list with a reason each**. Not copies: two files that resemble each other because they do similar jobs, which a similarity threshold cannot tell from a copy. **16 pairs.** The entry is a claim, and the claim is checked — the gate fails if a named pair ever becomes *identical* (that is a copy, and it says so), and fails if an entry stops matching anything at all, so stale justifications cannot accumulate.
- **unexplained** — everything else, and the only thing left that fails the gate. **0 pairs.**

How the 22 became 0, without running a board

13 of the 22 were real duplication and were removed. The other 9 were never duplication at all and are now named as such.

Removed — and proved to change nothing. This board's builds are deterministic (rebuilding an untouched tree gives byte-identical images, 19 of 19 on the Lyra), so "the images did not change" is a test, not a hope. Every removal below was made on its own and measured on its own:

removed	now lives in	images checked
`dma_pool.cpp`, `usb_env_stateos.cpp` — private copies in `smp_test6` and `smp_test7`	the Lyra board, as `UOS_BOARD_USB_GADGET_SOURCES`	19/19 Lyra **byte-identical**
`usb1_int3.cpp`, `kbd_forward.c` — a copy each in `smp_test_int3` and `smp_test_int4`	the Lyra board, as `UOS_BOARD_USB_INT_SOURCES` (`usb1_int3.cpp` only since 2026-09-26: `kbd_forward.c` is a host tool again, one copy in each test's `host/`, which the duplicate gate allows)	as above
`hw_result.hpp`, `board-contract.cpp` — one copy per ARM port	the kernel's `test_smpl/` (absorbed into `tests/smp-support/` on upstream integration — root `test_smpl/` is not carried into `xpack-development`)	96/96 Pi **byte-identical**, 19/19 Lyra
`syscalls.c` — one copy per `cortexm` board (4)	`cortexm/test/boards/shared/`	17/17 `cortexm` **byte-identical**

Two details made that possible, and both are the point rather than trivia. `board-contract.cpp` is named by each port *in the position its own copy held in the source list*, not attached as an INTERFACE source, because an INTERFACE source is appended elsewhere and moves link order. `syscalls.c` is inserted at the position the sorted glob used to find it, for the same reason. An earlier attempt that ignored this changed all 19 Lyra images — and that observation was then wrongly blamed on the USB change, which is what made this look like it needed hardware. It never did.

Named, not removed. The remaining 9 are in the script's `SIBLINGS` list, each with its reason: FatFs's `ff.c` against the Lyra's C++ `ff.cpp`; the RK3506 polled SD driver against the interrupt-driven variant `smp_test_int4` exists to exercise; the three `smp-test-nested-clock` pairs, which differ in N and B and in one working matrix versus two, with comments recording a measured PSRAM aliasing failure at N=500 on 2 MB; `cortexm`'s generic-M33 versus RP2350/generic `os-decls.h` and `os-c-decls.h` pairs; the single-core and SMP kernel-object tests; and TinyUSB's per-class `tusb_config.h`. Merging any of them would delete the difference that is the reason the file exists.

The gate was negative-tested three ways after the change, not just run against a clean tree: a newly introduced duplicate fails; a `SIBLINGS` pair made *identical* fails, and says that it is named but is now a copy; a `SIBLINGS` entry pointing at nothing fails as stale.

The absolute-path gate skips exactly two things: upstream's `tests/`, and **itself**. The second is not tidiness — its header comment spells every denied root out loud, so the moment it was committed it matched itself four times and could never pass again. A real absolute path added to that one file would go uncaught; that is the price of the denylist being readable, and it is the only exemption. Both gates have been negative-tested against an injected violation, before and after that change.

What the duplicate gate found, before it passed

Nothing in the kernel and nothing in a port's scheduler. This is the original list of 22, kept because it is the record of what the gate was for. 13 were removed and 9 are now named in `SIBLINGS`; see the previous section.

what	pairs	verdict
`aarch32`/`aarch64` `hw_result.hpp` and `board-contract.cpp`, byte-identical	2	**real** — candidates for the kernel repo
FatFs: `devices/fatfs/ff.c` vs the Lyra board's `fatfs-cpp/ff.cpp` (0.993, 5727 lines)	1	**real**, and the largest
RK3506 SD driver: `devices/soc/rk3506/sdmmc.*` vs `smp_test_int4`'s private copy	2	**real** — a test carrying its own driver
Lyra USB glue: the board's `usb/src/*` vs `smp_test6`/`smp_test7`'s copies	6	**real** — same shape as above
Lyra interrupt tests `smp_test_int3`/`int4` sharing host and USB sources	2	**real**
`cortexm` `syscalls.c`, four boards	3	**real**, small
RP2350 `smp-test-nested-clock{,_200,_250}`, differing only in a clock constant	3	a `board_test_defines()` job
`pico2-pizero` `sc-test-ko` vs `smp-test-ko` (0.912)	1	the single-core/SMP pair; arguably `board_test_ncpu()`
`cortexm` `os-decls.h` vs `include-rp2350/…` (0.936)	1	**known and deliberate** — the two port cores, see `cortexm-port.md`. Resolved 2026-09-25: identical by then, the RP2350 copy was removed
TinyUSB `tusb_config.h` cdc vs hid (0.852, 26 lines)	1	class-specific by design

Every one of them was in the Lyra's or `cortexm`'s test trees. None was in the kernel, in a port's scheduler, or in anything a passing suite exercises — which is why removing them could be, and was, verified by showing that every image came out byte-identical.

What hardware found that QEMU could not

Five real defects, all in paths the emulator does not reach:

Defect	Fix
Every blinking test drove **GPIO16**, so nothing lit	`LED_PIN` is a board fact now, 29 on the Zero 2 W, set once for every test. Only `usb_test` had ever set it — and the predecessor Makefiles had the same hole.
`usb_test` answered a PUT **after** streaming its hexdump and listing	`send_reply` moved to immediately after the store. A completed store looked like a hang.
Console posts **blocked** the USB service thread, so bulk OUT was not re-armed	`try_send`, with `console_dropped` in the tally. Diagnostics must never throttle the protocol.
`LIST` compared file names case-sensitively	FatFs on a boot card has no long names and returns `XFER.BIN`; flatfs under QEMU keeps the name as sent.
`kOutChunkMax` overflowed `PKTCNT` — a 1 MiB transfer stored 61440 bytes, silently truncated	`D{I,O}EPTSIZ` is bounded by two fields. `GHWCFG3 = 0x0ff000e8`: XFRSIZ 19 bits but PKTCNT **10 bits = 1023 packets**, so 65472 bytes at full speed. The chunk derives from the packet limit now, with `static_asserts` on both fields.

What made the test deduplication possible

The ISA seam in the applications turned out to be tiny, and mostly not a seam:

what the two copies disagreed on	resolution
`dsb` vs `dsb sy`, `dmb` vs `dmb ish`	not a difference — the explicit forms assemble on ARMv7-A too. Verified with both assemblers.
`cpsie i` vs `msr daifclr, #2`	already a kernel API: `interrupts::uncritical_section::enter()`, implemented by every port
`mrrc p15…c14` vs `mrs cntpct_el0`	already in the port's `timer_arm.hpp`; the tests were re-reading the register themselves
`"(AArch64)"` in banners	`PORT_BANNER_ISA` / `PORT_BANNER_CPU`, beside the existing `PORT_BANNER_LONG`/`SHORT`

Two pieces of per-application boilerplate were also collected: `test-smp-boot` (idle stacks, idle body, `smp_install_boot_threads()`, in ten of twelve tests) and `test-console` (the mutex-guarded print helpers, in three). The shared boot helper generates thread names from `OS_NCPU` instead of listing four, so it no longer assumes a four-core BCM2837.

Three defects found and fixed on the way

- **The kernel exported all 63 sources as one target**, but the proven rpi builds compiled exactly 37. Linking the rest breaks the build: `posix-io/c-syscalls-posix.cpp` declares `read`/`write` returning `ssize_t`, newlib declares them returning `int`. `micro-os-plus::iii` is now that measured 37-source core; posix-io, drivers, generic startup, newlib-reent, semihosting and the three trace backends are opt-in targets. Two entries were headers listed as sources and are gone — which is why 63 entries were always 61 compiled files.
- **The Cortex-A7 port's `os-decls.h` was missing `volatile`** on `lock_state[]`, which several cores read and write. The shared copy in `port/smp-common/` is the correct one, so adopting it fixes that port rather than merely deduplicating it.

- **`mutex::internal_try_lock_()` dereferenced a null owner.** In the priority-inheritance path it opens a `scheduler::uncritical_section` — which *releases* the kernel lock, because `priority_inherited()` ends in a yield — and then wrote through `owner_`. While that section is open the owner can finish its own `unlock()` on another CPU, and `internal_unlock_()` ends by setting `owner_` to `nullptr`. Every SMP port has the window; it is rare on hardware because `scheduler::unlock()`/`lock()` is a handful of instructions, but on the POSIX host it is two `pthread_sigmask()` system calls, so `smp_test2` faulted in 21 runs out of 25 at four CPUs. The fix captures the owner under the lock and re-checks ownership inside the uncritical section. **This is what a synthetic SMP host is for**, and it is the whole justification for step 5.

Decisions — all resolved

Spec Section 11 has the table; nothing is open. Revision 2 set the shape: architecture-specific ports (not target-specific), six repositories, architecture projects independent and *outside* the main repo, devices in their own repo, `cortexm` and `posix-arch` both to become SMP.

Next step

Step 2 is done. *Gate:* all 24 targets build **(met, both ports)**; QEMU suites pass **(met, 11/1/0 both ports)**; hardware tests pass on the Pi **(met, 12/12 both ports)**; no device or test source exists in more than one repo **(met)**.

Step 3 (`aarch32` gains the RK3506) is **under way**. *Gate:* `exception_handler.cpp` shared unmodified by both boards **(met)**; the board builds **(met, 19 targets)**; neither existing port regressed **(met, QEMU 11/1/0 on each)**. **The board has been run on hardware** — the maintainer confirms it, and this workspace retains one log of it, `test/build-lyra/test/.hw-logs/smp_test5.log`, which shows all three A7 cores up, the heartbeat running past 42 s and a clean shutdown. What is *not* recorded here is a per-test verdict table: the logs for the other eighteen targets are not in this tree, so this document does not claim pass/fail for them one by one. The earlier statement that the six RK3506-specific tests “have never been run” was wrong and is withdrawn.

Emulating the Lyra is **not** open. It was considered and closed: see above.

Step 4 (`cortexm`) is **under way**. *Gate:* no app source exists more than once; STM32 boards pass at `OS_NCPU=1`; pico2 passes at `OS_NCPU=2`. The repository exists and all six boards build — three STM32F4 (5 applications) and three RP2350 (36 images). **Nothing on this port had been run on hardware** when this was written (2026-09-22), so the two `pass` clauses of the gate were not met and were not claimed. Since then every board has a `-hwd` runner and hardware runs have been made (see the port’s git log); no per-test hardware verdict is recorded in this file. What the ELFs do show: the single-core tests carry no `launch_core1` and no per-core idle table while the SMP ones carry both, the PSRAM tests put `.bss` and the heap at `0x11000000` while their board-mates keep them in internal SRAM, each USB image carries only its own class driver, and the two kernel-less probes carry zero `os::rtos` symbols where their board-mates carry 145.

The spec framed this step as “merge pico2’s SMP core with the STM32 boards”. Measurement changed the shape, for the better:

- pico2’s core is **not portable Cortex-M SMP**. Its kernel lock is the RP2350’s SIO hardware spinlock 0, its IPI the SIO inter-core FIFO on IRQ 25, its CPU index `SIO_CPUID`. All three are silicon, so in this layout they are **board** code — and `cortexm` gained SMP by a board arriving, exactly as the Lyra’s GIC-400 SGI and the Pi’s are board code.

- The four “pico2 variants” differ in **one** thing, and it is that same kernel lock: `pico2` uses the SIO spinlock, `pico2-std` Peterson’s algorithm in software (the RP2350 has no global exclusive monitor, so LDREX/STREX does not work across the cores), `pico2-sdk` the Pico SDK — and `pico2-sdk-min` has **byte-identical** port files to `pico2-sdk`, so it is not a separate port at all. **Only `pico2` is of interest**; the other variants are out of scope and are not open work.

- **What is uncarried inside `pico2` is not a duplicate — it is a capability.** Of that tree's **37** application directories, `cortexm` carries 16: the kernel object tests, `smp-test0`…`smp-test5`, the nested-interrupt set and the two USB images. Of the **21** left, 20 are one subject area, and `cortexm` has none of its infrastructure (`find . -iname '*xip*' -o -iname '*loader*'` returns nothing but the PSRAM board files):

group	dirs	source lines
XIP: `smp-test-xip`, `-core1park`, `-fs`, plus `shared-xip` and `xip-glue`	5	7,454
XIP loader: `smp-test-xip-loader-{apps,apps-mos++,dyn,dyn-no-list,static}`	5	6,004
RAM loader: `smp-test-loader-{dynamic,over,reloc,static}`	4	1,527
pizero PSRAM execution: `pizero-psram-exec{,-main,-main_1,-main_2}`	4	1,551
`mem-diag`, `smp-test-nested_latency`	2	544

  17,080 lines in total, counting `.c`, `.cpp`, `.h`, `.hpp` and `.S` — the convention four of the five rows above were already written in. Counting every non-binary file as well (linker scripts, CMake, Makefiles, scripts, notes) gives 22,955, which is where this page's earlier "about 22,000" came from; the per-row figures were never on that footing. The twenty-first directory is `smp-test-mini-a-usb-cdc-acm_agy`, a variant of a USB test that *is* carried, and it belongs to no group above. Execute-in-place and a dynamic loader are a different piece of work from carrying another test, and neither was in step 4's gate. Recorded so the gap is not mistaken for a finished migration.

Step 5 (`posix-arch`) is **done**. *Gate:* the existing upstream tests pass at `OS_NCPU=1` **(met — `mutex-stress` *and* `rtos-apis`, both carried from the kernel's own `tests/sources/`; the nine SMP object tests cannot be built single-core, because they use `thread::cpu_affinity()`)**; the SMP object tests pass at `OS_NCPU>1` **(met, 9/9 at `OS_NCPU=4`)**; no use of `sigprocmask` or `setitimer` remains **(met — `pthread_sigmask` and per-CPU `timer_create`)**. Both ARM QEMU suites were re-run after the kernel mutex fix and after the shared test sources lost their last ARM assembly: 11/1/0 on each, unchanged. All four board/port pairs were run again after the dangling-name fix, one suite at a time: 11/1/0 on every one.

That the port really is SMP is **measured, not claimed**: four CPU-bound threads with no affinity set report 40 runs each on cores 0, 1, 2 and 3, and `smp_test4`'s eight workers each accumulate thousands of runs on all four CPUs. The counter-example matters as much — the same four threads with a `sleep_for()` in the loop report core 0 every time, because only CPU 0 advances the kernel clock. See the cold-start facts below before reading anything into a per-core distribution.

Sanitizers on the POSIX port

Measured, not asserted — the whole eleven-test suite was built and run under each. `-DUOS_SANITIZE=<set>` on the `native` board; details in `posix-arch-port.md` §21.

sanitizer	suite	reports	usable as a gate?
`address`	11/11 pass	0	**yes**
`undefined`	11/11 pass	29, all `vptr`, all in the kernel	**yes**, with `-DUOS_SANITIZE_NO_VPTR=ON` → 0 reports
`thread`			**no, and it cannot** — see below

sanitizer	suite	reports	usable as a gate?
	runs, still reaches `RESULT: PASS`	5,980 (3,775 races, 2,203 `errno`)	

Three things a fresh session should not have to rediscover:

- **The port is annotated for the ASan fiber API** and has to be. Without `__sanitizer_start_switch_fiber` / `__sanitizer_finish_switch_fiber` around `swapcontext`, every context switch is a false `stack-buffer-overflow`. The save slot is a **local**, so it rides the outgoing thread's own stack — a per-CPU slot would be read by the wrong host thread after a migration, the same hazard as the deferred publish.
- **ASan's first run found a real bug, and it is upstream.** `smp-pro-cons-test` named its eight workers from a loop-local `char name[16]`, and `os::rtos::named_object` stores `const char* const name_` — the pointer, never a copy. `scheduler::is_thread_allowed_on_cpu()` then `strcmp()`s that dead stack slot on **every scheduling decision**. It is in `micro-os-plus-iii-smp-old` (`rpi/…/64b/smp-pro-cons-test/main.cpp:524`, `:540`) and therefore in all **six** shipped copies of the test — `aarch32` × {rpi3b, rpi-zero-2w, luckfox-lyra}, `aarch64` × {rpi3b, rpi-zero-2w}, and `posix-arch/native`. **All six are now fixed**, and the five ARM copies are still byte-identical to each other. Four emulated boards rebuilt and re-run, `smp-pro-cons-test` green on all four; the Lyra copy builds and its image carries the fix, but it is hardware and was not run. A sweep found no second instance — `pool_thread_names`, `solver_thread_names` and `test-smp-boot.cpp`'s `idle_name[]` were already static.
- **The 29 UBSan reports are the kernel's intrusive-list sentinel idiom**, not a defect: `clock_timestamps_list::link()` downcasts the bare `head_` links to `timeout_thread_node*` to use as a loop terminator and only ever asks it for `->prev()`. Upstream already wraps those casts in `#pragma GCC diagnostic`. If that count ever changes, something else changed. Also: `-DCMAKE_CXX_FLAGS=-fno-sanitize=vptr` does **not** suppress them — `CMAKE_CXX_FLAGS` is emitted before the target's options and the last `-f[no-]sanitize=` wins. That is why there is a `UOS_SANITIZE_NO_VPTR` option.

TSan is closed, not open, and this is the thing not to re-attempt. The fiber annotation that looks like the fix — `__tsan_create_fiber` per thread, `__tsan_switch_to_fiber` before every `swapcontext`, the same shape of work that made ASan usable — was written, and it crashes: non-deterministically, inside `libtsan`, at `OS_NCPU=4` **and** at `OS_NCPU=1`, with and without the switches taken from the tick handler. Four configurations, four SEGVs.

The fault is not in the port. `posix-arch/tools/tsan-fiber-probe.c` is forty lines with no µOS++ in them — one fiber, two host threads, parked on A and resumed on B — and TSan stops with an internal assertion:

```
ThreadSanitizer: CHECK failed: tsan_rtl_proc.cpp:46
    "((thr->proc1)) == ((nullptr))"
```

`ProcWire()`: the fiber is still wired to host A's `Processor`. **TSan's fiber model is N:1**, many fibers on one host thread, built for coroutine libraries whose fibers stay put. This port is M:N by construction — migration is what `smp_test4` exists to demonstrate — so the two are structurally incompatible. The annotation was reverted rather than shipped behind an option that could only crash; the probe was kept, and answers in one second whether a future toolchain has changed its mind.

The `errno` half **is fixed**, separately and on its own evidence. `errno` belongs to the thread, and here the thread is the µOS++ one, not the host thread it is borrowing; the tick and IPI handlers now read it into a local on entry and put it back after their epilogue returns — the local rides the interrupted thread's own stack, and the restore goes through a `[[gnu::noinline]]` helper so no TLS address is carried across the switch point (which §19 of the port doc bans). **2,203 reports to 0**, measured.

The first attempt at it failed and the reason is the useful part: it put the save and restore in `switch_stacks()` instead of the handlers, and removed none of them, because TSan compares `errno` at handler **entry** against handler **exit** and `switch_stacks()` runs long after entry. The place was wrong, not the idea.

Things a fresh session should not rediscover

- **The RP2350 SIO spinlocks survive a re-flash.** Neither `reset init` nor a new image clears them; only a power cycle or the release loop in the board's `boot.S` does. Core 0 spinning in `_smp_klock_raw_acquire()` with the klock owner still `0xffffffff` is this.
- **A USB gadget thread on the RP2350 must be pinned at construction** (`th_cpu_affinity`), not by `cpu_affinity()` afterwards; see `cortexm/test/boards/pico2/usb/README-tinyusb.md`. Late pinning is a race: it passes, then a later run fails to enumerate.
- **On the Lyra, semihosting during USB traffic loses the debug link.** Each trap halts the core; the USB tests are built `UOS_CONSOLE_UART_ONLY` and report only their verdict through semihosting.
- **A dead UART on the psram board is the probe first.** On 2026-09-26 its CMSIS-DAP's UART bridge corrupted every received byte (break and framing errors in UART0 RSR) while the firmware was sound.
- **`smp-test-nested-clock*` are interactive:** they wait for `y` on the UART.
- **Two RP2350 tests compile no kernel**, and that is the point of them: `smp-test0` (dual-core bring-up) and `exc-test` (first-exception probe) run before a scheduler exists. `uos_add_app()` takes `NO_KERNEL`, the port exports a board-only `micro-os-plus::cortexm-bare` beside the full target, and `BOARD_TEST_NO_KERNEL` names them. Do not rewrite them as RTOS tests.
- **The port exports generic QEMU cores** beside the board ones: `micro-os-plus::cortexm-qemu-m0/-m3/-m4f/-m7` (with `::cortexm-qemu` as an alias of the M7 core) and `micro-os-plus::cortexm-qemu-m33`. The harness's `qemu-cortex-*`, `pico2-1cpu` and `2xcortex-m33` platforms link them with `tests/device-qemu-cortexm` at `OS_NCPU=1` (or `2` for `2xcortex-m33`).
- **On the RP2350 boards, SMP is a per-TEST fact, not a board fact.** `smp-test1` (single-core RTOS bring-up) and `sc-test-ko` (single-core kernel-object test) define `OS_USE_SMP_SCHEDULER` in no Makefile. `uos_add_app()` derives that define from `NCPU`, and `board_test_ncpu()` names the two tests. `sc-test-ko` is the only test on that silicon exercising the kernel's **non-SMP** branch.
- **The nested-interrupt tests need `-g0 -mlong-calls` on `main.cpp` alone.** They place handlers in `.data` to run from RAM instead of XIP flash. Without those two options the assembler stops with `Error: leb128 operand is an undefined symbol`. A call from flash to a `.data` function is also out of a Thumb `BL`'s reach.
- **`pico2-pizero` builds a different port branch.** It forces `__ARM_ARCH_8M_MAIN__` where the other two RP2350 boards force `__ARM_ARCH_7EM__`. The M33 is ARMv8-M Mainline and the predecessor's Makefiles for that board said so; the two branches have never been compared on the desk.
- **TinyUSB is carried once.** The predecessor kept `src/usb-cdc/` and `src/usb-hid/` byte-identical apart from one class driver each. The per-test include directory must come first — TinyUSB includes `tusb_config.h` by plain name.
- **`smp-test-mini-a-usb-cdc-acm_agy` is a byte-identical duplicate** of its sibling, `main.cpp` and `Makefile` both. Like `pico2-sdk-min`, it is not a separate thing.
- The four patched kernel copies are **byte-identical** (`diff -rq`).
- The kernel differs from pristine upstream in exactly **6 files**, all under `rtos/`. That is the entire SMP delta.
- **Upstream fork point**, preserved because the spec drops `BASE`: `micro-os-plus-iii` @ `c47f806b57f8b9b870ec7b89ded663dec354df96` (branch `xpack-development`), `micro-os-plus-iii-cortexm` @ `687e975caa298519cb214d4b5a9774c48f784816`.

- **`src/rtos/os-core.cpp` exists twice on purpose.** The kernel's is the portable SMP scheduler; each port has its own at the same relative path with the bring-up, the kernel lock and the context-switch hooks. Both are compiled, exactly as the old Makefiles did. They do not collide.
- **A port needs its machine flags at link time too**, not just compile time. Without them the driver picks the wrong multilib and every AArch32 link fails with "uses VFP register arguments".
- **Rebuild the port you are about to test.** A shared test source is still two separate binaries. `usb_test` on AArch32 reproduced a defect that had already been fixed, purely because only the AArch64 target had been rebuilt; the symptom was indistinguishable from the original bug.
- **A hardware run is one test per power cycle.** It is `load_image` into RAM over the previous test's leftovers, and the Pi has no SRST — the Cortex-A53 debug target has no reset method a script can drive. `run-hw.sh` refuses a suite for this reason.
- **A hardware budget is not the test's own duration.** It is dominated by semihosting traps, which scale with how much a test prints. `smp_test4` reaches its verdict at t=9597 ms of target time and still needs more than 120 s of wall clock.
- **`Invalid ACK (0) in DAP response` is not a firmware fault.** The debug link gave up; once it is gone nothing services the semihosting traps, so every core halts inside one and it reads like a hang. Drop the JTAG clock with `UOS_HW_ADAPTER_KHZ=1000` and check the supply. The runner reports this as DEBUG LINK LOST.
- **QEMU suites must run one at a time.** Running two four-core suites concurrently, or alongside a build, starves a vCPU: `smp_test2` stalled with three workers finished and the fourth frozen mid-loop. On an idle host the same binary passes 5 out of 5. A stalled worker with no fault is contention, not a bug — re-run before investigating.
- **The AArch32 suite runs on `raspi3b` with the boot shim, never `raspi2b`.** The port is built `-mcpu=cortex-a53`, so its load-acquire instructions are undefined on the raspi2b Cortex-A7 and it faults at the first one. QEMU's raspi3b starts its cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image. Hardware needs none of this: `config.txt` sets `arm_64bit=0`.
- **`rtk`'s `diff` reports "Files are identical" for files with different MD5s.** Seen on `sd.cpp`, `bcm2837.hpp`, `bcm_irq.hpp`. Use `md5sum` or Python `difflib` for anything that matters.
- `rtk` silently dropped the `-u` flag from `git push -u`; tracking was set with `git branch --set-upstream-to`. Expect this on new branches.
- Do **not** add a blanket `*.html` ignore rule — the kernel ships three doxygen templates as `.html`.
- There are **214 tracked Makefiles**, not the 24 an early count suggested.
- A test app is only ~5 tracked files; the ~100 others per directory are untracked build artifacts.
- **The SMP port contract is small.** The whole kernel patch is 379 lines and asks a port for only: `OS_USE_SMP_SCHEDULER`, `OS_NCPU`, `port_cpu_id`, `port::scheduler::switch_stacks`, `port::stack::element_t`, plus a kernel lock and an IPI.
- **`cortexm` SMP is not from scratch, and it is not portable.** pico2's `os-core-rp2350.cpp` is a complete dual-core Cortex-M33 SMP port, but its lock, IPI and CPU index are RP2350 SIO registers. It is 91% the same file as upstream's single-core core, and its port headers already handle every M profile (`6M`, `7M`, `7EM`, `8M_MAIN`) with the SMP parts gated on `OS_USE_SMP_SCHEDULER`. They are **not merged**: the three STM32 boards are hardware-proven on upstream's core, so a board names the core it was proven with (`UOS_BOARD_PORT_CORE`).

- **`pico2` forces `-D__ARM_ARCH_7EM__`** although the M33 is ARMv8-M, exactly as the predecessor's Makefiles did. The v7E-M path is the one that board was brought up on; letting the compiler pick `__ARM_ARCH_8M_MAIN__` would change the scheduler under a working board.

- **pico2's `smp-test0` compiles no kernel.** Nine files, none of them µOS++ — it is the bare-metal dual-core bring-up test that runs before any scheduler exists. It was first parked in `test/pico2/.pending/`, because `uos_add_app()` always linked `micro-os-plus::iii`; it now builds as a `BOARD_TEST_NO_KERNEL` application against the port's bare target, and `.pending/` is gone.

- **Flashing the Pico 2: resume cm1 BEFORE cm0.** With `USE_SMP 0` the rp2350 target exposes two targets and `reset init` halts both, while one `resume` resumes only the current one. A debug-halted core 1 will not boot from the PSM reset `launch_core1()` issues, so the launch hangs forever on the bootrom readiness word. Also `reset init`, never a bare `reset` — that does not re-run the bootrom/XIP setup and the new image never boots.

- **The STM32 boards are the only boards whose every image is `OS_NCPU=1`.** The harness's `pico2-1cpu` and `qemu-cortex-m0/m3/m4f/m7f` platforms also build the kernel's non-SMP branch at `OS_NCPU=1`, as do `pico2`'s `sc-test-ko` and `smp-test1`.

- **`posix-arch` was pristine upstream v1.0.1** — single host thread, `ucontext` coroutines, cooperative only — and is now a preemptive SMP port. The three POSIX defects D12 names are all fixed: `pthread_sigmask` for `sigprocmask`, per-CPU `timer_create` + `SIGEV_THREAD_ID` for `setitimer(ITIMER_REAL)`, and **native TLS is banned** rather than threads being pinned — no port or application state may live in `errno` or a `thread_local` across a switch point.

- **The SMP picker reads `th->context_.port_.stack_ptr` directly**, and the design spec does not say so. Null means "not safe to claim". The POSIX port puts `stack_ptr` first in its thread context and uses it as the same flag, published not by the CPU that leaves a thread but by **whoever arrives next on that CPU** — because by the time `swapcontext()` returns, this CPU is already running the incoming thread. Same hazard and same answer as AArch64's `_smp_pub_addr`/`_smp_pub_val`; no kernel change was needed.

- **On the POSIX host the tick handler runs with no `SA_ONSTACK`**, on the µOS++ thread's own stack, so the signal frame migrates with the thread. That is why `OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES` is 32 KiB there. The *fault* handler is the opposite case and does use `SA_ONSTACK`.

- **The POSIX free store is `first_fit_top`, not glibc `malloc`.** A µOS++ thread can be preempted inside an allocation and resumed on a different host thread, and glibc's arena lock would then be released by a thread that never took it.

- **`hw_result::ok()`/`fail()` flush stdio before `_exit()` on the host.** `_exit()` does not, and to a pipe stdout is fully buffered — a carried upstream test ran, passed, and printed nothing at all.

- **The POSIX fault reporter prints the faulting pc, the image's load bias and a backtrace**, so `addr2line -e <image> <pc-bias>` names the line. It is how the mutex race above was found, after three speculative fixes had resolved nothing.

- **Wake-ups on the POSIX host are biased towards CPU 0, and that is not a bug.** Only CPU 0 calls `os_systick_handler()` — the BCM2837 arrangement, where all four cores take the 1 ms PPI but one advances the kernel clock — so a thread released by `sleep_for()`, a timer or a timeout is normally re-picked by CPU 0 before any other CPU's tick comes round. **CPU-bound work spreads evenly; clock-bound and I/O-bound work does not.** Measured: four CPU-bound threads report 40 runs each on c0/c1/c2/c3 with no affinity set anywhere, while the same four with a `sleep_for()` in the loop report core 0 every time. Both are correct. This is also why `smp-pipeline-test` prints `Produced by Core: c0=3275 c1=0 c2=0 c3=0`, and why `smp_test4` — whose workers are CPU-bound — shows every worker on every CPU. **Before suspecting the scheduler because a test reports all its work on core 0, check whether its threads**

are CPU-bound or clock-bound. Distributing the clock across CPUs would change this and would also stop the port resembling the silicon it stands in for; do not “fix” it.

- **On the POSIX host, install the per-core idle threads BEFORE releasing the cores**: `smp_install_boot_threads()` then `smp::start_secondary_cores()`, never the reverse. A released CPU enters `reschedule()` at once, and with no idle thread of its own it finds nothing to run and aborts with `!!! no ready thread and no idle thread on CPU 1 !!!`. Every carried test already does it in that order.